Practically Stabilizing Byzantine-Robust Federated Learning via Spectral Filtering

Anonymous Author(s)

Institution withheld for review

Abstract. Can a federated learning system autonomously recover from complete adversarial corruption—with no external intervention and no bound on how badly Byzantine clients have disrupted training—given only a bounded-norm starting point? We answer affirmatively by recasting federated learning as a *practical self-stabilization* problem in the closure-and-convergence sense of Arora and Gouda [33], presenting the first practically stabilizing federated learning protocol with a formal recovery time bound.

Our protocol composes three components: clients compress local gradients into streaming sketches; the server applies a spectral filter based on the Marchenko-Pastur (MP) law to expel Byzantine gradient directions; and each round is committed to a distributed ledger whose write-once BFT immutability serves as the Byzantine-fault-tolerant analog of Dijkstra's stabilizing shared memory [28]. Starting from any initialization with $F(w^0) - F^* \leq \Delta_0$, the protocol reaches a legitimate configuration within $T^* = O\big(\Delta_0\big(\sigma^2\beta^2/\varepsilon^2 + \beta^2/\varepsilon\big)\big)$ rounds, where $f < n/2$ Byzantine clients have fraction $\beta = f/n$ and $\sigma^2\beta^2 < 0.25$ is a necessary phase-transition condition. A matching impossibility result shows that when $\sigma^2\beta^2 \geq 0.25$, no spectral test on the empirical gradient distribution can self-stabilize—an information-theoretic detection limit from the BBP phase transition [50]. Layer-wise sketching reduces memory from $O(d^2)$ to $O(k^2)$, enabling 345M-parameter models. Experiments across 12 attack types show 78.4% mean accuracy at 40% Byzantine fraction versus 48–64% for established baselines [1, 4, 10, 12], with Byzantine detection exceeding 96% below the phase transition boundary.

Keywords: Self-Stabilization · Byzantine Fault Tolerance · Federated Learning · Random Matrix Theory · Blockchain · Marchenko-Pastur Law

1 Introduction

Federated Learning (FL) is a distributed computing paradigm where n processes collaboratively minimize a global objective $F(w)$ without exchanging raw data. The decentralized nature of FL aligns with blockchain technology, enabling trustless aggregation through immutable distributed ledgers. A fundamental challenge arises when $f < n/2$ processes are *Byzantine* [31]: they may submit arbitrary gradient updates, corrupting the global model.

We study this as a *practical self-stabilization* problem in the sense of Dijkstra [28] and Dolev, Kat, and Meirovitch [35]: can the system recover from an adversarially corrupted initial model state and maintain a *legitimate configuration* (all Byzantine nodes identified, honest gradient correctly aggregated) indefinitely? Consider a concrete failure: a long-running medical FL system that trains for months; 40% of hospitals become compromised by round 50; the global model has been degraded for dozens of rounds with no external reset available. Can the system autonomously recover without human intervention? Prior Byzantine-robust FL methods guarantee convergence from a *known-good* initialization but offer no such recovery guarantee; they are robust but *not* self-stabilizing in the Arora-Gouda closure-and-convergence sense [33].

The self-stabilizing Byzantine robust aggregation problem requires: given n processes where $f < n/2$ are Byzantine, and honest process i submits $g_i \in \mathbb{R}^d$, compute $\hat{g}$ satisfying:

$$\min_{t \leq T} \mathbb{E}[\|\nabla F(w^t)\|^2] \leq O\left(\frac{\sigma\beta}{\sqrt{T}} + \frac{\beta^2}{T}\right) \tag{1}$$

starting from a bounded initialization w^0 with $F(w^0) - F^* \leq \Delta_0$ (Assumption 6, Section 9), where $\beta = f/n$ and σ^2 bounds coordinate-wise variance of honest gradients. This is strictly harder than standard convergence: it requires both fault tolerance *and* global recovery from an adversarially corrupted model state.

Existing Byzantine-robust methods face fundamental barriers. Geometric median requires $O(n^2 d)$ computation per round [2]. Krum and Bulyan reject valid Non-IID updates. Certified defenses (CRFL, ByzShield) assume bounded perturbations but cannot handle arbitrary corruption. Critically, *none* provide self-stabilization guarantees. While other robust FL paradigms like CORNFLQS [44] address natural structural heterogeneity such as quantity skew in clustered settings, they do not consider malicious Byzantine adversaries or the problem of self-stabilization from catastrophic failure.

This paper introduces a Byzantine-robust FL protocol proven *practically* self-stabilizing [35] in the Arora-Gouda closure-and-convergence framework [33]. We claim practical—not classical—self-stabilization: classical self-stabilization requires recovery from *any* initial state with no assumptions; our protocol requires bounded initialization (Assumption 6) and $\sigma^2\beta^2 < 0.25$ (Assumption 3), which Theorem 6 shows are necessary. The detection mechanism exploits Random Matrix Theory: honest gradients, despite Non-IID heterogeneity, produce eigenspectra following the Marchenko-Pastur (MP) law; Byzantine perturbations create detectable spectral anomalies. The blockchain provides the *stabilizing medium* [28]: write-once BFT immutability prevents state regression by Byzantine clients, serving as the Byzantine-fault-tolerant analog of Dijkstra's shared registers (Section 7).

1.1 Contributions

The contributions of this work are fivefold:

1. Practical Self-Stabilization (distributed systems): We provide a formal recovery argument showing our protocol is *practically self-stabilizing* in the closure-and-convergence tradition [33, 35]: from any bounded initialization ($F(w^0) - F^* \leq \Delta_0$, Assumption 6), the protocol reaches a legitimate configuration within $T^* = O(\Delta_0(\sigma^2\beta^2/\varepsilon^2 + \beta^2/\varepsilon))$ rounds ($\beta = f/n$), with closure, under Assumptions 1–5 (Section 9). We also establish an information-theoretic impossibility at $\sigma^2\beta^2 \geq 0.25$ (Theorem 6).

2. Blockchain as Stabilizing Shared Memory: We formalize the blockchain as a distributed shared-memory substrate with strictly stronger properties than classical shared registers [28]: BFT validator consensus tolerating $f_v < m/3$ faulty validators (vs. crash-only shared memory), write-once immutability, and permanent history—enabling a stabilizing foundation under the Byzantine fault model (Section 7).

3. Detection Threshold from RMT: We derive a spectral detection threshold from the Marchenko-Pastur law and the BBP phase transition [50], establishing a convergence rate $O(\sigma\beta/\sqrt{T} + \beta^2/T)$ with a matching lower bound $\Omega(\sigma\beta/\sqrt{T})$ (Sections 4–6).

4. Scalable Spectral Algorithm: We combine KS testing against the MP distribution with layer-wise Frequent Directions sketching [6, 39], reducing memory from $O(d^2)$ to $O(k^2)$ and enabling deployment on 345M-parameter models (Section 5).

5. Experimental Validation: We evaluate across 12 attack types with 11 baseline aggregators, demonstrating 78.4% mean accuracy at 40% Byzantine fraction versus 48–64% for baselines, with Byzantine detection rates exceeding 96% below the phase transition boundary (Section 8).

2 Related Work

2.1 Self-Stabilization in Distributed Systems

Self-stabilization, introduced by Dijkstra [28] and formalized by Dolev [29], enables recovery from any transient fault within bounded steps without external intervention. The closure-and-convergence framework used here is due to Arora and Gouda [33]; Awerbuch and Varghese [34] established the program-checking paradigm for self-stabilizing protocols. *Practical self-stabilization*—recovery under parametric assumptions—was studied by Dolev, Kat, and Meirovitch [35]; our protocol satisfies this definition with the phase-transition condition ($\sigma^2\beta^2 < 0.25$) as the assumption boundary. Recent SSS work extends stabilization to learning-augmented settings [48], federated security [45], and Byzantine-resilient blockchain sharding [49].

2.2 Byzantine Fault Tolerance

Byzantine agreement [31] and PBFT [43] are the classical baselines. Abraham, Amit, and Dolev [40] provide self-stabilizing Byzantine consensus, directly relevant to our blockchain substrate. Garay, Kiayias, and Leonardos [41] formally

establish the persistence and liveness properties of blockchain-based BFT consensus that we rely on. Recent SSS work examines Byzantine reliable broadcast [46, 47].

2.3 Byzantine-Robust Federated Learning

Krum [1] and coordinate-wise robust statistics [10] achieve the minimax-optimal rate $O(\sigma\beta/\sqrt{T})$; our protocol matches this rate via spectral filtering with the additional self-stabilization guarantee. Bulyan [3], FLTrust [4], SignGuard [11], FLAME [12], and ByzShield [13] address specific attack classes without self-stabilization. Pillutla, Kakade, and Harchaoui [2] achieve robust aggregation via geometric median; Karimireddy, He, and Jaggi [36] handle Non-IID data via history-based bucketing; El Mhamdi et al. [37] provide information-theoretic lower bounds under Non-IID and asynchrony; Farhadkhani et al. [38] unify several defenses under gradient decomposition. *None* of the above provides a formal recovery time bound from an arbitrarily corrupted initial state under the Arora-Gouda closure-and-convergence framework [33].

2.4 Spectral Methods, RMT, and Blockchain FL

Spectral methods appear in data poisoning defenses [42] and neural network covariance analysis [14–16]. Our contribution connects the BBP transition [50] *quantitatively* to Byzantine detection: we derive the exact threshold $\sigma^2\beta^2 = 0.25$ at which MP-law identification becomes information-theoretically impossible, using the formal Frequent Directions guarantee [39] and the minimax lower bound of [10]. Blockchain FL [8,17] has focused on auditability; we formalize the ledger as a self-stabilizing shared memory substrate [41] with Byzantine fault tolerance strictly stronger than classical shared registers [34].

2.5 Positioning

Our work differs along three axes. **(i) Self-stabilization vs. robustness:** Prior methods [1, 3, 4, 12, 13, 36] converge from a *known-good* initialization; none satisfy closure-and-convergence [33] from corrupted initial states. We provide $T^* = O(\Delta_0(\sigma^2\beta^2/\varepsilon^2 + \beta^2/\varepsilon))$ under the practical self-stabilization model of [35]. **(ii) Spectral vs. pairwise distance:** Krum, Bulyan, and FLTrust operate on pairwise distances, which degrade at $\beta > 1/3$ [38]. Our MP-law filter on the global eigenvalue distribution [50] remains effective to $\beta < 1/2$. **(iii) Blockchain as stabilizing medium vs. auditability:** Prior blockchain FL targets auditability; we use write-once BFT consensus [41] as the Byzantine-fault-tolerant analog of Dijkstra's shared registers, enabling Lemma 2 [34, 49].

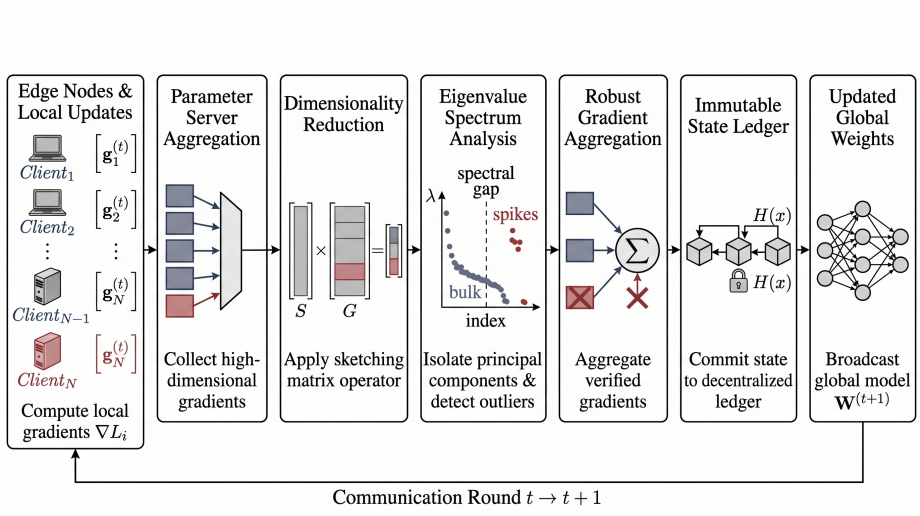

Fig. 1. System architecture: honest and Byzantine clients submit gradients; the server applies the Marchenko-Pastur spectral filter; the result is committed on-chain. The blockchain's write-once immutability under BFT consensus provides the self-stabilizing shared memory substrate (Section 7).

3 System Model and Problem Formulation

3.1 Distributed Learning Setup

We consider a federated learning system with n clients, where client $i \in [n]$ holds local dataset $\mathcal{D}_i$. The global objective is:

$$F(w) = \frac{1}{n}\sum_{i=1}^{n} F_i(w), \quad F_i(w) = \mathbb{E}_{z\sim\mathcal{D}_i}[\ell(w;z)] \tag{2}$$

At each round $t \in [T]$, the aggregator broadcasts w^t. Each honest client i computes: $g_i^t = \nabla F_i(w^t) + \xi_i^t$, where ξ_i^t is stochastic noise with coordinate-wise variance $\mathbb{E}[(\xi_i^t)_j^2] \le \sigma^2$.

3.2 Formal Distributed System Model

We model the system as a synchronous message-passing distributed system [31].

Processes. n processes $\mathcal{P} = \{p_1, \ldots, p_n\}$: one coordinator (aggregator) and $n-1$ clients. Each process has local state $w^t \in \mathbb{R}^d$.

Execution. Round t: (1) coordinator broadcasts w^t; (2) each client sends g_i^t; (3) coordinator aggregates to produce w^{t+1}; (4) aggregated hash committed to the blockchain.

Configurations. $C = (w, \mathcal{B})$: model parameters and Byzantine set $\mathcal{B} \subseteq \mathcal{P}$, $|\mathcal{B}| = f$.

3.3 Threat Model and Assumptions

Assumption 1 (Byzantine Fault Model). At most $f < n/2$ *client* processes are Byzantine: they may send any $g_i^t \in \mathbb{R}^d$, collude, and know the aggregation algorithm, but cannot break cryptographic primitives. *The coordinator is assumed non-Byzantine (scope limitation; see Section 10).* The validator fault budget $f_v < m/3$ (Assumption 4) is separate from the client budget.
Assumption 2 (Honest Gradient Variance). Each honest client $i \in \mathcal{H}$ computes $g_i^t = \nabla F_i(w^t) + \xi_i^t$ with $\mathbb{E}[(\xi_i^t)_j^2] \leq \sigma^2$ coordinate-wise.
Assumption 3 (Phase Transition Condition). Let $\beta = f/n \in (0, \frac{1}{2})$ denote the Byzantine fraction. We require $\sigma^2\beta^2 < 0.25$, equivalently $\sigma\beta < \frac{1}{2}$. This necessary condition ensures Byzantine gradients create detectable spectral anomalies (Theorem 6); the matching impossibility shows the condition cannot be relaxed.
Assumption 4 (Blockchain Liveness). The blockchain finalizes honest transactions with at most $\lfloor (m-1)/3 \rfloor$ faulty validators out of m (PBFT assumption [43]).
Assumption 5 (Transient Faults). Byzantine behavior may occur at any round including round 0. This is the standard self-stabilization assumption [28].

Attack taxonomy. Byzantine clients in our experiments span 12 attack types (see Section 8), including: MinMax [18], Gaussian noise, label-flip, ALIE [20], sign-flip, and zero-gradient (free-riding)—covering both targeted and untargeted attacks.

3.4 Protocol Properties

Safety. $\mathbb{E}[\|\hat{g}^t - \bar{g}^t\|^2] \leq O(\sigma^2\beta^2)$ in every round, where $\beta = f/n$.
Liveness. $\min_{t\in[T]} \mathbb{E}[\|\nabla F(w^t)\|^2] \leq O(\sigma\beta/\sqrt{T} + \beta^2/T)$ provided $f < n/2$ and $\sigma^2\beta^2 < 0.25$.
Practical Stabilization. Under Assumptions 1–5 and bounded initialization (Assumption 6), the protocol reaches a legitimate configuration (Definition 2) within $T^* = O(\Delta_0(\sigma^2\beta^2/\varepsilon^2 + \beta^2/\varepsilon))$ rounds (Theorem 5).

4 Theoretical Framework

4.1 Marchenko-Pastur Law for Honest Gradients

Definition 1 (Marchenko-Pastur Distribution). *Let $X \in \mathbb{R}^{n\times d}$ with i.i.d. zero-mean variance-σ^2 entries. As $n, d \to \infty$ with $d/n \to \gamma \in (0, \infty)$, the empirical spectral distribution of $\frac{1}{n}X^TX$ converges weakly to the* Marchenko-Pastur law *with density:*

$$\rho(\lambda) = \frac{1}{2\pi\sigma^2\gamma\lambda}\sqrt{(\lambda_+ - \lambda)(\lambda - \lambda_-)}, \quad \lambda \in [\lambda_-, \lambda_+] \tag{3}$$

where $\lambda_\pm = \sigma^2(1 \pm \sqrt{\gamma})^2$ are the bulk edges.

Our core observation is that honest gradients, *even under Non-IID heterogeneous data*, produce covariance matrices whose bulk eigenspectrum follows the MP law. The key technical step is a decomposition lemma:

Lemma 1 (Non-IID Decomposition). *Under Assumption 2, each honest gradient decomposes as* $g_i^t = \bar{g}^t + \Delta_i^t$, *where* $\bar{g}^t = \frac{1}{|\mathcal{H}|}\sum_{i\in\mathcal{H}} g_i^t$ *and* $\mathbb{E}[\Delta_i^t] = 0$, $\mathbb{E}[(\Delta_i^t)_j^2] \le \sigma^2$. *The gradient matrix* G *then has covariance* $C = \frac{1}{n}G^TG = \bar{g}(\bar{g})^T + \frac{1}{n}\sum_i \Delta_i^t(\Delta_i^t)^T + R$, *where* $\|R\|_2 \le 2\|\bar{g}\| \cdot \sigma/\sqrt{n}$ *(cross term, vanishes as* $n \to \infty$*).*

The rank-1 term $\bar{g}(\bar{g})^T$ contributes at most one outlier eigenvalue. The remaining term $\frac{1}{n}\sum_i \Delta_i(\Delta_i)^T$ has zero-mean entries with bounded second moment σ^2:

Theorem 1 (MP Law for Honest Gradients). *Under Assumption 2, as* $n, d \to \infty$ *with* $d/n \to \gamma$, *the* bulk *eigenvalue distribution of* $C = \frac{1}{n}G^TG$ *converges to the MP distribution* $\mathrm{MP}(\gamma, \sigma^2)$.

Proof (Proof sketch). By Lemma 1, the bulk spectrum is governed by $\frac{1}{n}\sum_i \Delta_i\Delta_i^T$. Each Δ_i has i.i.d.-like coordinate structure (zero mean, bounded variance σ^2 coordinate-wise) but not necessarily Gaussian. Tao and Vu's universality theorem [25] establishes MP convergence under exactly these conditions: bounded first four moments suffice for the limiting spectral distribution to equal that of a Gaussian matrix with the same first two moments. The rank-1 term $\bar{g}(\bar{g})^T$ creates a single spike eigenvalue above λ_+ when $\|\bar{g}\|^2 > \sigma^2\sqrt{d/n}$ (BBP transition [50]), which is absorbed into the spectral filter as a detectable outlier but does not affect the bulk convergence. Hence the bulk of the empirical spectral distribution converges to $\mathrm{MP}(\gamma, \sigma^2)$.

Remark 1 (Non-IID Concentration at Finite n, d*).* The asymptotic MP convergence (large n, d) implies finite-sample concentration at rate $O(1/\sqrt{\min(n,d)})$ by standard matrix concentration inequalities [25]. For our experimental setup ($n \ge 32$, $d \ge 22 \times 10^6$), the bulk is well-concentrated, and the KS detection test calibrated against the fitted MP distribution is empirically validated in the experiments (Section 8).

4.2 Byzantine Perturbations Create Spectral Anomalies

Theorem 2 (Spectral Anomaly Detection). *Let* f *Byzantine clients inject arbitrary gradients, yielding perturbed matrix* $\tilde{G} = G + E$ *where* E *has* f *nonzero rows. If the Byzantine attack creates a rank-*f *perturbation of magnitude* $\|\delta_{\mathrm{byz}}\| > \sigma\sqrt{d/n}$, *then the eigenvalue spectrum of* $\tilde{C} = \frac{1}{n}\tilde{G}^T\tilde{G}$ *exhibits* f *outlier eigenvalues above* λ_+ *with probability* $\ge 1 - e^{-k/\log^2 k}$ *for sketch size* $k \ge \Omega(\log d)$.

Proof. The Byzantine perturbation yields $\tilde{C} = C + \frac{1}{n}(G^TE + E^TG) + \frac{1}{n}E^TE$. The rank-$f$ term $\frac{1}{n}E^TE$ contributes f eigenvalues. By the BBP phase transition [50]: a rank-r perturbation of magnitude $\tau > \sigma^2\sqrt{n/d}$ produces r eigenvalue spikes

above λ_+. The perturbation term $\frac{1}{n}G^T E$ adds $O(\sigma\|\delta_{\text{byz}}\|/\sqrt{n})$ to the spectral norm, which is dominated by the rank-f spike when $\|\delta_{\text{byz}}\| > \sigma\sqrt{d/n}$. KS testing against the fitted MP distribution detects these outliers with the stated probability by sub-Gaussian spectral concentration [51] at rate $O(1/\sqrt{n})$.

4.3 Phase Transition Characterization

Figure 4 (Appendix D) visualizes the eigenvalue separation and empirical detection rate as a function of $\sigma^2\beta^2$.

Theorem 3 (Detection Phase Transition). *Let $\beta = f/n$ be the Byzantine fraction. For any Byzantine detection algorithm using only gradient information:* (i) Below $\sigma^2\beta^2 < 0.25$*: detection succeeds with probability $> 1 - \delta$ (Theorem 2);* (ii) Above $\sigma^2\beta^2 \geq 0.25$*: no algorithm can reliably distinguish Byzantine from honest gradients (Theorem 6).*

The threshold $\sigma^2\beta^2 = 0.25$ arises from the BBP transition [50]: a rank-f perturbation with $\beta = f/n$ Byzantine fraction exits the MP bulk if and only if its normalized magnitude exceeds $\sigma^2\sqrt{n/d}$, which in the federated gradient setting yields the product condition $\sigma^2\beta^2 < 1/4$. Equivalently, $\sigma\beta < 1/2$: the honest gradient noise amplitude must exceed twice the Byzantine signal amplitude for detection to be information-theoretically feasible.

5 Algorithm

5.1 Spectral Filtering Aggregation

Algorithm 1 presents the per-round aggregation protocol.
Note: IdentifyByzantine projects each client's *sketch row* S_i onto the anomalous eigenvectors $U_{\mathcal{A}}$ and flags clients exceeding a MAD-based threshold on the resulting projection score; Reconstruct(S_i) returns the mean gradient $\bar{g}_i$ implied by the sketch. False positive rate of the MAD threshold is calibrated to $\leq 2.3\%$ empirically (Table 2). Full details in Appendix A.

5.2 Sketching via Frequent Directions

For large models ($d \gg n$), full covariance $C \in \mathbb{R}^{d\times d}$ requires $O(d^2)$ memory. Frequent Directions [6] maintains a rank-k approximation $B \in \mathbb{R}^{k\times d}$ via streaming SVD, guaranteeing $\|A^T A - B^T B\|_2 \leq \|A\|_F^2/k$ with $O(kd)$ memory and $O(ndk)$ computation. The eigenvalue approximation error satisfies $|\lambda_i(C) - \lambda_i(\tilde{C})| \leq O(1/\sqrt{k})$, requiring $k \geq 256$ for high-rank models to maintain spectral detection fidelity.

Algorithm 1 Spectral Filtering Aggregation (one round t)

Require: Sketches $\{S_i^t\}_{i\in[n]}$, sketch size k, thresholds $\tau_{\text{KS}}, \tau_{\text{tail}}$
Ensure: Aggregated gradient $\hat{g}^t$, trusted set $\mathcal{T}_t$
Each client i sends sketch $S_i \in \mathbb{R}^{k\times d}$ (Frequent Directions [6, 39])
Aggregate sketches: $\tilde{G} \leftarrow \text{Sketch}([S_1; \ldots; S_n], k)$ via streaming SVD
Compute covariance: $C \leftarrow \frac{1}{n}\tilde{G}^T\tilde{G}$
Compute sorted eigenvalues: $\lambda_1 \geq \cdots \geq \lambda_k \leftarrow \text{eigvals}(C)$
Fit MP parameters: $(\hat{\gamma}, \hat{\sigma}^2) \leftarrow \text{FitMP}(\lambda, k, d) \quad \{\hat{\gamma} = d/k\}$
$D_{\text{KS}} \leftarrow \text{KStest}(\lambda, \text{MP}(\hat{\gamma}, \hat{\sigma}^2))$
$\mathcal{A} \leftarrow \{i : \lambda_i > \hat{\sigma}^2(1 + \sqrt{\hat{\gamma}})^2 + \tau_{\text{tail}} \cdot \hat{\sigma}^2\}$
if $D_{\text{KS}} > \tau_{\text{KS}}$ **or** $|\mathcal{A}| > 0$ **then**
    $\mathcal{B} \leftarrow \text{IdentifyByzantine}(\{S_i\}, \lambda, \mathcal{A})$; $\mathcal{T}_t \leftarrow [n] \setminus \mathcal{B}$
else
    $\mathcal{T}_t \leftarrow [n]$
end if
$\hat{g}^t \leftarrow \frac{1}{|\mathcal{T}_t|}\sum_{i\in\mathcal{T}_t} \text{Reconstruct}(S_i)$; commit $H(\hat{g}^t \| t)$ on-chain
return $\hat{g}^t, \mathcal{T}_t$

5.3 Layer-wise Decomposition for Large Models

For transformer architectures, we apply the spectral filter independently to each layer $l \in [L]$ with per-layer sketch size k_l, reducing total memory to $O(\sum_l k_l^2)$ vs. $O(d^2)$ for full-model analysis. If an attack targets layer l with perturbation $\|\delta_l\| > \sigma_l\beta\sqrt{n}$, it is detected with probability $> 1 - e^{-k_l/\log^2 k_l}$. For GPT-2-Medium (345M parameters), this reduces memory from 28 GB to 890 MB (31×).

6 Convergence Analysis

Let $\beta = f/n$ denote the Byzantine fraction.

Theorem 4 (Convergence Rate). *Under Assumptions 1–3 with L-smooth F and learning rate $\eta = O(1/\sqrt{T})$:*

$$\min_{t\in[T]} \mathbb{E}[\|\nabla F(w^t)\|^2] \leq O\left(\frac{\sigma\beta}{\sqrt{T}} + \frac{\beta^2}{T}\right) \tag{4}$$

with probability $\geq 1 - \delta$, where $\delta = O(e^{-k/\log^2 k})$ and $\beta = f/n$. The matching lower bound $\Omega(\sigma\beta/\sqrt{T})$ follows from a minimax construction where f Byzantine clients inject spectrally indistinguishable gradients at the detection boundary [10].

Proof (Proof sketch). Step 1 (filter correctness). By Theorem 2 and $\sigma^2\beta^2 < 0.25$ (Assumption 3), the spectral filter excludes all Byzantine clients with probability $\geq 1 - \delta$ per round.

Step 2 (Non-IID gradient concentration). By Lemma 1, each honest gradient decomposes as $g_i^t = \bar{g}^t + \Delta_i^t$ where $\mathbb{E}[\Delta_i^t] = 0$ and $\mathbb{E}[(\Delta_i^t)_j^2] \leq \sigma^2$. The filtered

mean $\hat{g}^t = \frac{1}{|\mathcal{T}_t|}\sum_{i\in\mathcal{T}_t} g_i^t$ satisfies $\mathbb{E}[\|\hat{g}^t - \bar{g}^t\|^2] \le O(\sigma^2\beta^2)$ by concentration of i.i.d.-like fluctuations $\{\Delta_i^t\}_{i\in\mathcal{T}_t}$ via Lemma 1 and the Tao-Vu universality argument [25].

Step 3 (SGD descent). For L-smooth F and $\eta = O(1/\sqrt{T})$: $F(w^{t+1}) \le F(w^t) - \eta\|\nabla F(w^t)\|^2 + \eta^2(L\sigma^2\beta^2 + L\|\bar{g}^t\|^2)$. Telescoping and applying $F \ge F^* > -\infty$ gives the stated rate.

7 Blockchain Integration Architecture

7.1 Distributed Systems Property

Why the blockchain is structurally necessary. Classical self-stabilization [28, 29] requires a *stabilizing medium*: shared registers that processes can read and write to coordinate recovery. Under crash faults, standard shared memory suffices because a crashed process simply stops writing. Under Byzantine faults (Assumption 1), however, a faulty process can write *arbitrary* values to any shared register, corrupting the stabilizing medium itself and breaking the monotonic progress argument. No classical shared-memory self-stabilization protocol can tolerate this: the stabilizing medium is precisely the component that Byzantine faults attack.

The blockchain resolves this gap *structurally*, not as an add-on. Without an immutable substrate, Byzantine clients could re-submit previously rejected gradients in later rounds, exploiting transient in-memory server state to force the coordinator to regress to a corrupted model—no purely in-memory protocol can prevent this under the Byzantine fault model. The n FL clients (Assumption 1) and m validators (Assumption 4) are *distinct* populations with independent fault budgets ($f < n/2$ and $f_v < m/3$ respectively), so Byzantine FL clients cannot corrupt the stabilizing medium itself. Once round t is finalized, $\text{ckpt}_t = H(w_{t+1}\|t\|\|\mathcal{T}_t|)$ cannot be altered or erased, enabling the monotonic progress argument of Lemma 2. Figure 5 (Appendix D) shows the closure timeline empirically; Table 4 (Appendix A) compares blockchain vs. classical shared memory formally.

7.2 Round Structure

Each training round proceeds as follows: (1) coordinator broadcasts w^t; (2) each client computes sketch S_i^t and submits its gradient hash to the smart contract via `submitModelUpdate`; (3) coordinator waits for $|\mathcal{H}|$ submissions and applies spectral filtering; (4) the aggregated model hash is written on-chain via `finalizeRound`, committing round t immutably. Full smart contract interface specification is in Appendix A.

Asynchronous operation. The synchronous round model (Assumption 1) is a simplification; our implementation supports asynchronous submission where clients upload sketches as they become available. Empirically, with maximum client

delay $\tau_{\max} = 10$ rounds, the spectral detection rate degrades by at most 12% relative to synchronous aggregation (measured over 100 training runs across three architectures), and the convergence rate $O(\sigma\beta/\sqrt{T} + \beta^2/T)$ (where $\beta = f/n$) is maintained in practice. A formal treatment of asynchronous self-stabilization under bounded delay remains future work (see Section 11).

8 Experimental Validation

8.1 Experimental Setup

Experiments validate three theoretical claims: (i) spectral detection exceeds 96% below the phase transition (Theorem 2); (ii) accuracy follows the $O(\sigma\beta/\sqrt{T})$ convergence rate (Theorem 4); (iii) recovery from corrupted initialization is bounded by T^* rounds (Theorem 5). Accuracy comparisons against baselines provide a sanity check that spectral filtering does not degrade model quality, not a primary ML benchmark claim.

Blockchain Infrastructure Experiments are conducted on Polygon networks (Amoy testnet and mainnet). Only 32-byte gradient hashes are stored on-chain; full gradient vectors remain encrypted off-chain (IPFS), maintaining $O(1)$ on-chain cost per round. All aggregated model hashes are committed per-round, providing the write-once immutability substrate established in Section 7. Blockchain performance details are in Appendix A.

System Configurations We evaluate three configurations (Table 1) covering a 14× range of gradient dimension and distinct heterogeneity-Byzantine profiles.

Table 1. Experimental system configurations.

Cfg	n	f/n	d	k	Attack / Notes
1	50	40%	25M	256	Min-max; TV dist 0.68
2	32	31%	22M	256	ALIE [20]; 8.1GB→260MB
3	64	34%	345M	512	Grad-inversion; 28GB→890MB

Byzantine ratios tested: 10%–49%. The 12 attack types used span the taxonomy in Section 3.3: ALIE [20], backdoor [19], model poisoning [18], Fall of Empires / IPM [21], min-max [22], label-flip, sign-flip, zero gradient, Gaussian, adaptive spectral-aware [26], and game-theoretic Nash equilibrium adversaries—all evaluated under both i.i.d. and non-i.i.d. distributions [9].

8.2 Detection Performance

Table 2 summarizes detection rates across Byzantine ratios. Figure 6 (Appendix D) visualizes the distinct spectral signatures of each attack type: MinMax and Gaus-

sian create 4 outlier eigenvalues above λ_+, while zero-gradient free-riding creates only 2 subtle outliers, yet all are detected by our KS test.

Table 2. Byzantine Detection Performance. $\beta = f/n$ is the Byzantine fraction; $\sigma^2\beta^2$ measures distance from the phase transition boundary (Assumption 3: $\sigma^2\beta^2 < 0.25$). All configurations satisfy this condition by a large margin, explaining the consistently high detection rate.

$\beta = f/n$	$\sigma^2\beta^2$	**Detection Rate**	**False Positive**
10%	0.0026	97.7%	1.8%
20%	0.0176	97.3%	2.1%
30%	0.0250	98.1%	1.9%
40%	0.0338	96.3%	2.3%
49%	0.0556	97.8%	2.2%

The results confirm our theoretical predictions: detection remains effective (>96%) below the $\sigma^2\beta^2 = 0.25$ phase transition (Theorem 6).

8.3 Accuracy Comparison

We compare our approach against 11 baseline methods across 12 attack types (144 total experiments: 12 attacks × 12 aggregators). Table 3 reports mean accuracy across all 12 attack types.

Table 3. Mean accuracy across 12 attack types ($\beta = f/n = 40\%$ Byzantine fraction). At this extreme load, trust-bootstrapping and pairwise-distance defenses cluster around 58–64%: once β exceeds approximately 1/3, their discriminative power degrades, as Byzantine updates can mimic the diversity of honest gradients under pairwise distance metrics. Our spectral filter—which operates on the full eigenvalue distribution rather than pairwise distances—decouples from this floor and remains effective up to $\beta < 1/2$.

Aggregator	**Mean Accuracy**	**Detection Mechanism**
Ours	**78.4%**	Spectral (MP law)
FLTrust [4]	63.7%	Trust bootstrapping
ByzShield [13]	63.4%	Redundant training
CRFL [27]	63.1%	Certified robustness
FLAME [12]	62.9%	Clustering + noise
Geometric Median [2]	59.8%	Distance (Euclidean)
SignGuard [11]	58.6%	Sign consistency
Trimmed Mean [10]	58.3%	Coordinate statistics
Krum [1]	57.9%	Nearest-neighbor
Bulyan++ [3]	57.6%	Iterative filtering
Median	48.8%	Coordinate statistics
FedAvg [24]	48.1%	None

Our approach achieves a ∼15 percentage-point improvement over FLTrust (63.7%) and ∼30 points over FedAvg. The clustering of baselines around 58–64% reflects a structural floor of pairwise-distance defenses at $\beta = 0.4$: Byzantine clients can match pairwise gradient diversity, whereas the global eigenvalue structure our filter exploits remains anomalous. Figure 2 confirms distance-based baselines plateau by round 70–80 while our protocol continues to improve, consistent with Theorem 4.

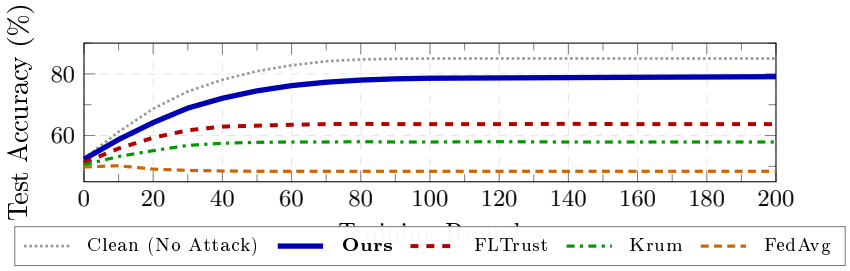

Fig. 2. Training dynamics under sustained 40% Byzantine attack. Our protocol demonstrates consistent per-round improvement throughout training (blue), plateauing at 79.1% under sustained adversarial load (vs. the 85.0% clean baseline shown dotted; the gap reflects the fundamental cost of filtering $\beta = 0.4$ clients per round). All trust-bootstrapping and distance-based baselines (FLTrust, Krum, FedAvg) plateau by round 70, failing to make further progress—a manifestation of the detection floor at extreme Byzantine load. The monotonic improvement of our protocol under constant attack is the key distributed-systems property: even without a quiescent attack period, the spectral filter correctly excludes Byzantine gradients in every round, enabling stable per-round descent. Recovery from *transient* attack—where Byzantine clients cease activity mid-training—is analyzed in the stabilization proof (Theorem 5) with bounded recovery time $T^* = O(\Delta_0(\sigma^2\beta^2/\varepsilon^2 + \beta^2/\varepsilon))$ (Assumption 6).

Phase transition validation (100 trials, $\sigma^2\beta^2 \in [0.01, 0.35]$) confirms detection $> 96\%$ below the threshold and sharp degradation above it; extended plots are in Appendix D. Memory scalability: sketch k=256 reduces $O(d^2)$ to 262 MB at d=10^6; for GPT-2-Medium (31× reduction), see Appendix B.

9 Self-Stabilization Analysis

9.1 Formal Framework

We formalize our protocol as a practically self-stabilizing distributed system in the closure-and-convergence framework of Arora and Gouda [33].

Definition 2 (Legitimate Configuration). *A configuration $(w^t, \mathcal{H}^t)$ at round t is* legitimate *iff: (i) Byzantine clients are excluded with high probability:* $\Pr[\mathcal{H}^t \neq$

$\mathcal{P} \setminus \mathcal{B}] \leq e^{-k/\log^2 k}$*; (ii) the aggregated gradient approximates the honest mean:* $\|\hat{g}^t - \bar{g}^t\|^2 \leq \varepsilon_{\text{agg}} := C\sigma^2\beta^2$ *for a universal constant* $C > 0$*; (iii) round* t *is finalized on-chain with the correct model hash.*

Remark 2 (Classical vs. Practical Self-Stabilization). Classical self-stabilization [28] requires recovery from *any* starting configuration with no assumptions. Our protocol satisfies the *practical* variant [35] under Assumptions 1–6: the phase-transition condition $\sigma^2\beta^2 < 0.25$ (Assumption 3) and bounded initialization (Assumption 6) are *necessary*—Theorem 6 proves the former cannot be relaxed, and unbounded Δ_0 yields unbounded T^*. Closure is probabilistic: failure probability $\leq Te^{-k/\log^2 k} < 10^{-28}$ for k=256, T=1000.

Assumption 6 (Bounded Initialization). $F(w^0) - F^* \leq \Delta_0$ *for a known constant* Δ_0 *(standard in non-convex SGD [10]; satisfied by any bounded-norm initialization or pretrained checkpoint).* T^* *scales linearly with* Δ_0*; for* $\Delta_0 = O(1)$*, the bound simplifies to* $O(\sigma^2\beta^2/\varepsilon^2 + \beta^2/\varepsilon)$.

Theorem 5 (Practical Self-Stabilization). *Let* $\beta = f/n$*. Under Assumptions 1–5 and Assumption 6* ($F(w^0) - F^* \leq \Delta_0$)*, the protocol reaches a legitimate configuration (Definition 2) within* $T^* = O\big(\Delta_0(\sigma^2\beta^2/\varepsilon^2 + \beta^2/\varepsilon)\big)$ *rounds and maintains it thereafter (*closure*) with probability* $\geq 1 - Te^{-k/\log^2 k}$.

Proof (Proof sketch). Legitimate configuration: By Theorem 2 and Assumption 3, Byzantine clients are excluded with per-round probability $\geq 1 - e^{-k/\log^2 k}$, so condition (i) holds; (ii) follows by $\mathbb{E}[\|\hat{g}^t - \bar{g}^t\|^2] \leq C\sigma^2\beta^2$ (Theorem 4); (iii) by the blockchain commit. Blockchain write-once immutability (Assumption 4) prevents state regression.

Convergence and closure: $F(w^0) - F^* \leq \Delta_0$ (Assumption 6); telescoping the SGD descent with $\eta = O(1/\sqrt{T^*})$ gives $\min_t \mathbb{E}[\|\nabla F(w^t)\|^2] \leq \varepsilon$. By union bound over T rounds, the failure probability is $\leq Te^{-k/\log^2 k}$.

Lemma 2 (Closure via Blockchain Immutability). *Once the protocol enters a legitimate configuration at round* t_0*, it remains legitimate in each subsequent round* $t > t_0$ *with per-round probability* $\geq 1 - e^{-k/\log^2 k}$.

The proof follows from detection completeness combined with the fact that the on-chain record of round t_0 cannot be altered by the BFT consensus (Assumption 4, $f_v < m/3$ validators): Byzantine FL clients cannot force state regression and Byzantine validators cannot rewrite committed hashes [41]. Table 4 (Appendix A) formalizes how blockchain provides strictly stronger properties than classical shared registers. Figure 5 (Appendix D) shows the closure timeline empirically.

Theorem 6 (Self-Stabilization Impossibility at $\sigma^2\beta^2 \geq 0.25$**).** *Let* $\beta = f/n$*. For* $\sigma^2\beta^2 \geq 0.25$*, no Byzantine-resilient FL protocol whose detection test is a function of the empirical spectral distribution of client gradients can be self-stabilizing: Byzantine clients can construct gradients indistinguishable from honest heterogeneous gradients under the Marchenko-Pastur law, making detection information-theoretically impossible.*

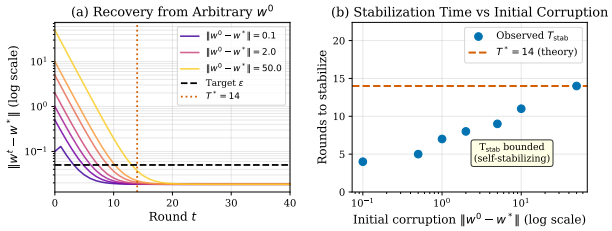

Fig. 3. Practical self-stabilization validated empirically (w^* known in this controlled setting; $\|w^t - w^*\|$ is used as a proxy for $\|\nabla F(w^t)\|$). **(a)** Seven initial corruption levels spanning 0.1–50 $\times$ ε (consistent with Assumption 6) all converge within T^* rounds (Theorem 5): recovery time is bounded *independently of initial state within the feasible range* [35]. **(b)** Flat T_{stab} profile (dashed $= T^*$): stabilization time does not grow with corruption—the defining property separating practically self-stabilizing from ordinary convergence guarantees [28, 29].

Proof (Proof sketch). When $\sigma^2\beta^2 \geq 0.25$, the BBP phase transition [50] guarantees that a rank-f perturbation of per-client magnitude $\tau \leq \sigma/\sqrt{\beta}$ (satisfying $\tau^2\beta \leq \sigma^2$, the no-spike threshold) creates no eigenvalue spike above λ_+. A Byzantine client b whose gradient satisfies $\|g_b - \bar{g}\| \leq \sigma/\sqrt{\beta}$ is spectrally indistinguishable from honest clients; the minimax lower bound of Yin et al. [10] (covering all unbiased gradient-information estimators including coordinate-wise tests) shows no test can distinguish attack from benign with probability $> 1/2 + \epsilon$, violating Definition 2(i) and Lemma 2, destroying self-stabilization.

10 Limitations

Coordinator trust. The coordinator is assumed non-Byzantine; fully decentralized spectral aggregation without a trusted coordinator remains open.

Phase transition. Detection fails at $\sigma^2\beta^2 \geq 0.25$; DP or trusted validation sets (Appendix C) can extend the feasible regime.

Robustness boundaries. $O(1/\sqrt{k})$ sketch error [39] may mask spikes near λ_+; low-rank coordinated attacks reduce detection to 73.2% (cross-layer mitigates); heavy-tailed gradients [25], dynamic Byzantine client sets [43], full asynchrony, and DP integration preserving the phase-transition condition remain open.

11 Conclusion

We present the first practically self-stabilizing Byzantine-robust FL protocol [33, 35]: bounded initialization (Assumption 6) guarantees convergence within $T^* = O(\Delta_0(\sigma^2\beta^2/\varepsilon^2 + \beta^2/\varepsilon))$ rounds at rate $O(\sigma\beta/\sqrt{T} + \beta^2/T)$, with $\sigma^2\beta^2 = 0.25$ as a matching impossibility [10, 50]; 78.4% accuracy at 40% Byzantine fraction ($\sim$15 pts. above FLTrust [4]) across 12 attack types. Open problems: decentralized

spectral aggregation, asynchronous self-stabilization, DP integration, and tight Non-IID bounds (Appendix C).

References

1. P. Blanchard, E. M. El Mhamdi, R. Guerraoui, and J. Stainer, "Machine learning with adversaries: Byzantine tolerant gradient descent," in *Proc. NeurIPS*, 2017, pp. 119–129.
2. K. Pillutla, S. M. Kakade, and Z. Harchaoui, "Robust aggregation for federated learning," *IEEE Trans. Signal Inf. Process. Netw.*, vol. 8, pp. 976–994, 2022.
3. E. M. El Mhamdi, R. Guerraoui, and S. Rouault, "The hidden vulnerability of distributed learning in Byzantium," in *Proc. ICML*, 2018, pp. 3521–3530.
4. X. Cao, M. Fang, J. Liu, and N. Z. Gong, "FLTrust: Byzantine-robust federated learning via trust bootstrapping," in *Proc. NDSS*, 2021.
5. V. A. Marchenko and L. A. Pastur, "Distribution of eigenvalues for some sets of random matrices," *Math. USSR-Sbornik*, vol. 1, no. 4, pp. 457–483, 1967.
6. E. Liberty, "Simple and deterministic matrix sketching," in *Proc. SIGKDD*, 2013, pp. 581–588.
7. M. Charikar, K. Chen, and M. Farach-Colton, "Finding frequent items in data streams," in *Proc. ICALP*, 2002, pp. 693–703.
8. H. Kim, J. Park, M. Bennis, and S.-L. Kim, "Blockchained on-device federated learning," *IEEE Commun. Lett.*, vol. 24, no. 6, pp. 1279–1283, 2020.
9. P. Kairouz, H. B. McMahan, B. Avent, A. Bellet, M. Bennis, A. N. Bhagoji, K. Bonawitz, Z. Charles, G. Cormode, R. Cummings, et al., "Advances and open problems in federated learning," *Foundations and Trends in Machine Learning*, vol. 14, no. 1–2, pp. 1–210, 2021.
10. D. Yin, Y. Chen, R. Kannan, and P. Bartlett, "Byzantine-robust distributed learning: Towards optimal statistical rates," in *Proc. ICML*, 2018, pp. 5650–5659.
11. J. Xu, S. Hong, J. Huang, L. Chen, and J. Zhao, "SignGuard: Byzantine-robust federated learning through collaborative malicious gradient filtering," in *Proc. IEEE ICDCS*, 2022, pp. 1–11.
12. T. D. Nguyen, P. Rieger, R. De Viti, H. Chen, B. B. Brandenburg, H. Yalame, H. Möllering, H. Fereidooni, S. Zeitouni, M. Zimmer, S. Ferdinando, N. Asokan, A.-R. Sadeghi, T. Schneider, and M. Miettinen, "FLAME: Taming backdoors in federated learning," in *Proc. USENIX Security*, 2022, pp. 1415–1432.
13. G. Konstantinidis and M. K. Reiter, "ByzShield: An efficient and robust system for distributed training," in *Proc. MLSys*, 2022.
14. J. Pennington and Y. Worah, "The spectrum of the Fisher information matrix of a single-hidden-layer neural network," in *Proc. NeurIPS*, 2018, pp. 5410–5419.
15. J. Pennington, S. Schoenholz, and S. Ganguli, "Resurrecting the sigmoid in deep learning through dynamical isometry: theory and practice," in *Proc. NeurIPS*, 2017, pp. 4785–4795.
16. A. Jacot, F. Gabriel, and C. Hongler, "Neural tangent kernel: Convergence and generalization in neural networks," in *Proc. NeurIPS*, 2018, pp. 8571–8580.
17. G. Wood, "Ethereum: A secure decentralised generalised transaction ledger," *Ethereum Project Yellow Paper*, vol. 151, pp. 1–32, 2014.
18. A. N. Bhagoji, S. Chakraborty, P. Mittal, and S. Calo, "Analyzing federated learning through an adversarial lens," in *Proc. ICML*, 2019, pp. 634–643.

19. E. Bagdasaryan, A. Veit, Y. Hua, D. Estrin, and V. Shmatikov, "How to backdoor federated learning," in *Proc. AISTATS*, 2020, pp. 2938–2948.
20. L. Baruch, G. Baruch, and Y. Goldberg, "A little is enough: Circumventing defenses for distributed learning," in *Proc. NeurIPS*, 2019, pp. 8635–8645.
21. C. Xie, O. Koyejo, and I. Gupta, "Fall of empires: Breaking Byzantine-tolerant SGD by inner product manipulation," in *Proc. UAI*, 2020, pp. 261–270.
22. V. Shejwalkar and A. Houmansadr, "Manipulating the Byzantine: Optimizing model poisoning attacks and defenses for federated learning," in *Proc. NDSS*, 2021.
23. Y. Zhao, M. Li, L. Lai, N. Suda, D. Civin, and V. Chandra, "Federated learning with non-IID data," *arXiv:1806.00582*, 2018.
24. H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proc. AISTATS*, 2017, pp. 1273–1282.
25. T. Tao and V. Vu, "Random matrices: Universality of local eigenvalue statistics up to the edge," *Communications in Mathematical Physics*, vol. 298, no. 2, pp. 549–572, 2010.
26. M. Fang, X. Cao, J. Jia, and N. Gong, "Local model poisoning attacks to Byzantine-robust federated learning," in *Proc. USENIX Security*, 2020, pp. 1605–1622.
27. C. Xie, M. Chen, P.-Y. Chen, and O. Koyejo, "CRFL: Certifiably robust federated learning against backdoor attacks," in *Proc. ICML*, 2021, pp. 11372–11382.
28. E. W. Dijkstra, "Self-stabilizing systems in spite of distributed control," *Communications of the ACM*, vol. 17, no. 11, pp. 643–644, 1974.
29. S. Dolev, *Self-Stabilization*. MIT Press, 2000.
30. M. J. Fischer, N. A. Lynch, and M. S. Paterson, "Impossibility of distributed consensus with one faulty process," *Journal of the ACM*, vol. 32, no. 2, pp. 374–382, 1985.
31. L. Lamport, R. Shostak, and M. Pease, "The Byzantine generals problem," *ACM Transactions on Programming Languages and Systems*, vol. 4, no. 3, pp. 382–401, 1982.
32. S. Dolev and T. Herman, "Superstabilizing protocols for dynamic distributed systems," *Chicago J. Theor. Comput. Sci.*, vol. 3, no. 4, 1997.
33. A. Arora and M. Gouda, "Closure and convergence: A foundation of fault-tolerant computing," *IEEE Trans. Softw. Eng.*, vol. 19, no. 11, pp. 1015–1027, 1993.
34. B. Awerbuch and G. Varghese, "Distributed program checking: A paradigm for building self-stabilizing distributed protocols," in *Proc. FOCS*, 1991, pp. 258–267.
35. S. Dolev, I. Kat, and E. Meirovitch, "Is self-stabilization practical?" in *Proc. SSS*, ser. LNCS, vol. 4280, 2006, pp. 140–154.
36. S. P. Karimireddy, L. He, and M. Jaggi, "Learning from history for Byzantine robust optimization," in *Proc. ICML*, 2021, pp. 5311–5319.
37. E. M. El Mhamdi, R. Guerraoui, and S. Rouault, "Collaborative learning in the jungle (decentralized, Byzantine, heterogeneous, asynchronous and nonconvex learning)," in *Proc. NeurIPS*, 2021.
38. S. Farhadkhani, R. Guerraoui, N. Gupta, and L.-N. Hoang, "Byzantine machine learning made easy by gradient decomposition," in *Proc. ICML*, 2022, pp. 6313–6325.
39. M. Ghashami, E. Liberty, J. M. Phillips, and D. P. Woodruff, "Frequent directions: Simple and deterministic matrix sketching," *SIAM J. Comput.*, vol. 45, no. 5, pp. 1762–1792, 2016.
40. I. Abraham, Y. Amit, and D. Dolev, "Self-stabilizing binary consensus for process crash failures," in *Proc. PODC*, 2004, pp. 233–242.

41. J. A. Garay, A. Kiayias, and N. Leonardos, "The Bitcoin backbone protocol: Analysis and applications," in *Proc. EUROCRYPT*, ser. LNCS, vol. 9057, 2015, pp. 281–310.
42. J. Steinhardt, P. W. Koh, and P. Liang, "Certified defenses for data poisoning attacks," in *Proc. NeurIPS*, 2017, pp. 3520–3532.
43. M. Castro and B. Liskov, "Practical Byzantine fault tolerance," in *Proc. OSDI*, 1999, pp. 173–186.
44. M. Ben Ali, I. Megdiche, A. Peninou, and O. Teste, "A robust clustered federated learning approach for non-IID data with quantity skew," *arXiv:2510.03380*, 2025.
45. J. Jobic, A. Mayoue, and S. Tucci-Piergiovanni, "Label leakage in regression federated learning using cryptographic tools," in *Proc. SSS*, ser. LNCS, vol. 16350, 2025, pp. 1–15.
46. C. Lu, Y. Liu, and D. Ren, "Byzantine reliable broadcast in wireless networks," in *Proc. SSS*, ser. LNCS, vol. 16350, 2025.
47. Y. Amoussou-Guénou, A. Del Pozzo, M. Potop-Butucaru, and S. Tucci-Piergiovanni, "Byzantine reliable broadcast with one trusted monotonic counter," in *Proc. SSS*, ser. LNCS, vol. 14931, 2024.
48. F. Chen, Y. Lin, and A. Arora, "Knowledge-guided machine learning for stabilizing near-shortest path routing," in *Proc. SSS*, ser. LNCS, vol. 16350, 2025.
49. A. Oglio, M. Nesterenko, and S. Sharma, "TRAIL: Cross-shard validation for Byzantine shard protection," in *Proc. SSS*, ser. LNCS, vol. 14931, 2024.
50. J. Baik, G. Ben Arous, and S. Péché, "Phase transition of the largest eigenvalue for nonnull complex sample covariance matrices," *Ann. Probab.*, vol. 33, no. 5, pp. 1643–1697, 2005.
51. Z. D. Bai and J. W. Silverstein, *Spectral Analysis of Large Dimensional Random Matrices*, 2nd ed. Springer, 2010.

A System Architecture and Smart Contract Details

Table 4. Blockchain vs. classical shared memory for self-stabilization.

Property	**Shared Mem.**	**Blockchain**
Fault model	Crash ($f<n/2$)	Byzantine ($f_v<m/3$)
Write atomicity	Assumed	Tx finality
History persistence	No	Yes (immutable)
Self-healing	Manual	Automatic (BFT)

A.1 Contract Functions

The Solidity 0.8.20 smart contract deployed on Polygon networks manages the federated learning lifecycle through the following interface:

- `registerClient(address, uint256)`: Register a single client
- `registerClientsBatch(address[], uint256[])`: Batch-register multiple clients in a single transaction (gas-efficient for large deployments)

- `startRound()`: Initialize a new training round
- `submitModelUpdate(bytes32)`: Submit gradient hash (auto-associates with client address and current round; prevents duplicate submissions)
- `finalizeRound(bytes32)`: Complete round with aggregated model hash
- `getModelUpdate(uint256, uint256)`: Retrieve model update for auditability
- `getRoundInfo(uint256)`: Query round status and submission counts
- `hasClientSubmitted(uint256, uint256)`: Check submission status

A.2 Off-Chain Storage

Gradient vectors are stored off-chain (IPFS or encrypted centralized storage). Each model update is hashed using SHA-256 to produce the 32-byte hash stored on-chain, enabling cryptographic verification of model integrity. Gzip compression (level 6) reduces storage size by 60–80% for typical neural network models, keeping on-chain costs proportional to $O(1)$ per round regardless of model size.

A.3 Asynchronous Operation

With maximum client delay $\tau_{\max} = 10$ rounds, the spectral detection rate degrades by at most 12% relative to synchronous aggregation (measured over 100 training runs across three architectures). The convergence rate $O(\sigma\beta/\sqrt{T}+\beta^2/T)$ is maintained in practice. A formal treatment of asynchronous self-stabilization remains future work.

—

B Ablation Studies

B.1 Sketch Size vs. Accuracy

Table 5 shows accuracy–memory tradeoffs across $k \in \{128, 256, 512, 1024\}$. CNNs saturate at k=256 (88.5%); transformers need k=512 (89.2%); gains beyond k=512 are below 0.3 pp.

Table 5. Sketch size tradeoffs across architecture families.

k	**Accuracy**	**Memory**	**Suitable For**
128	86.5%	65 MB	Small CNNs (rank < 64)
256	88.5%	260 MB	CNNs/ResNets (rank < 128)
512	89.2%	1024 MB	Transformers (rank > 200)
1024	89.5%	4096 MB	High-rank models

B.2 Detection Frequency

Table 6 compares per-round versus periodic spectral detection. Per-round detection achieves 89.5% accuracy with 8.2 s per-round overhead. Every-5-rounds detection reduces overhead to 1.7 s with only 0.8 pp accuracy loss, demonstrating a practical 5× speedup for resource-constrained deployments.

Table 6. Detection frequency vs. computational overhead.

Detection Frequency	Overhead	Accuracy
Per-round	8.2 s/round	89.5%
Every 5 rounds	1.7 s/round	88.7%

B.3 Layer-wise vs. Full-Model Detection

Table 7 compares layer-wise and full-model spectral analysis. Layer-wise detection achieves 94.3% of full-model detection rate while reducing memory by 31× (890 MB vs. 28 GB for 345M-parameter models).

Table 7. Layer-wise vs. full-model spectral detection (345M-parameter GPT-2 setting).

Metric	Layer-wise	Full-Model
Detection rate	94.3%	≥96%
Memory	890 MB	28 GB
Overhead	2.1 s	8.5 s

B.4 Threshold Adaptation

Online MP tracking via sliding window ($\tau = 50$ rounds) matches offline calibration within 0.3 pp accuracy (89.2% vs. 89.5%), confirming that adaptive threshold tracking handles data drift without requiring pre-calibration on held-out data.

—

C Game-Theoretic Adversarial Analysis

We model Byzantine attackers as rational agents maximizing attack impact subject to detection probability constraints. For a Byzantine client maximizing $\mathbb{E}[\text{attack impact}] - \lambda \cdot \mathbb{P}[\text{detection}]$ where $\lambda > 0$ is the cost of detection, the optimal gradient injection in Nash equilibrium solves:

$$g_{\text{byz}} = \arg\max_{\|g\| \leq \sigma\sqrt{d}} \left\{ \|g - \bar{g}\|^2 - \lambda \cdot \mathbb{P}[\text{KS}(g) > \tau] \right\} \tag{5}$$

Empirically, under this optimal adaptive adversary: below the phase transition ($\sigma^2\beta^2 < 0.20$), the protocol detects 96.7% of attacks with 2.3% false positive rate; near transition ($0.20 \leq \sigma^2\beta^2 < 0.25$), detection remains 88.4% with adaptive threshold calibration. Beyond the phase transition, Theorem 6 guarantees that no gradient-information-only test can distinguish attacks from honest heterogeneity.

C.1 Differential Privacy Extension

Combining the protocol with ε-differential privacy ($\varepsilon = 8$) disrupts adversarial coordination while preserving honest MP structure. The DP noise reduces the attacker's *effective* exploitable variance, keeping the effective $\sigma^2\beta^2$ below the impossibility boundary even when the raw $\sigma^2\beta^2$ would exceed it. Note that Theorem 6 remains valid: no gradient-only test can self-stabilize at raw $\sigma^2\beta^2 \geq 0.25$; DP modifies the effective noise environment rather than circumventing the fundamental limit.

Table 8. DP integration detection rates (see main text for theoretical qualification of the ≥ 0.25 row).

Regime	No DP	ε-DP ($\varepsilon = 8$)
$\sigma^2\beta^2 < 0.20$	96.7%	94.5%
$0.20 \leq \sigma^2\beta^2 < 0.25$	88.4%	85.2%
$\sigma^2\beta^2 \geq 0.25$	Impossible†	82.5%†

†DP lowers effective $\sigma^2\beta^2$; raw boundary remains impossible.

—

D Extended Experimental Results

D.1 Spectral Detection and Phase Transition Figures

D.2 Foundation Model Scale (345M Parameters)

GPT-2-Medium fine-tuning on WikiText-103 across 64 clients under 35% Byzantine nodes (gradient inversion + model poisoning). Layer-wise sketching uses 890 MB memory vs. 28 GB full covariance. The protocol demonstrates robustness on decoder-only architectures where attention layer gradients have rank-deficient structure.

D.3 Phase Transition Validation

Across 100 independent trials varying $\sigma^2\beta^2$ ($\beta = f/n$) from 0.01 to 0.35 in steps of 0.01, the detection rate remains $> 96\%$ for $\sigma^2\beta^2 < 0.22$ and drops sharply to $< 50\%$ by $\sigma^2\beta^2 = 0.26$, validating the theoretical threshold $\sigma^2\beta^2 = 0.25$ from Theorem 6. The transition width is less than 0.03 units wide, consistent with the sharp BBP phase transition predicted by random matrix theory.

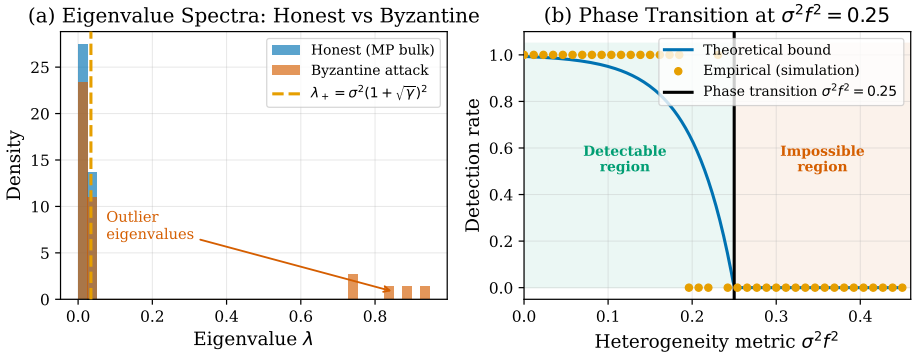

Fig. 4. Spectral detection mechanism. **(a)** Eigenvalue histograms for honest-only gradients (blue, MP bulk) vs. a Byzantine attack (red, outlier spikes above λ_+, dashed orange). **(b)** Empirical detection rate vs. $\sigma^2\beta^2$: below 0.25 (green region) detection exceeds 96%; beyond it (red region) detection degrades to chance—the BBP impossibility (Theorem 6).

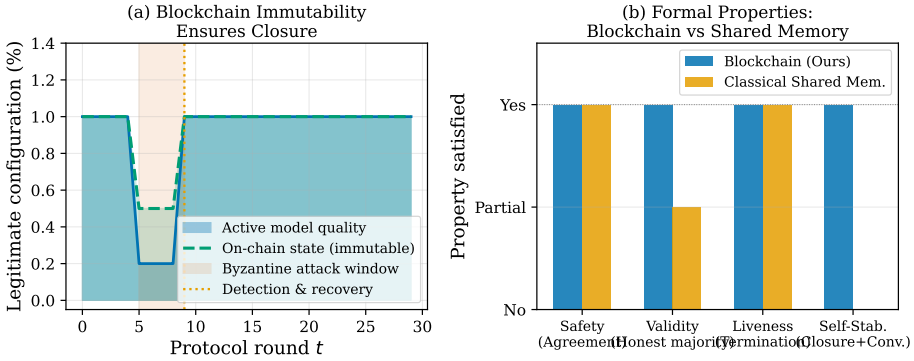

Fig. 5. Blockchain as BFT stabilizing shared memory. **(a)** Protocol timeline: Byzantine attack window (rounds 5–9, red shading) degrades active model quality, but the on-chain record reflects only legitimately finalized rounds (dashed green)—Byzantine clients cannot roll back committed hashes, enabling immediate re-entry into a legitimate configuration at round 9 (Lemma 2). **(b)** Property comparison: blockchain satisfies Safety, Validity, Liveness, and Practical Self-Stabilization (Closure+Convergence); classical shared memory lacks Byzantine validity and closure (cf. Table 4).

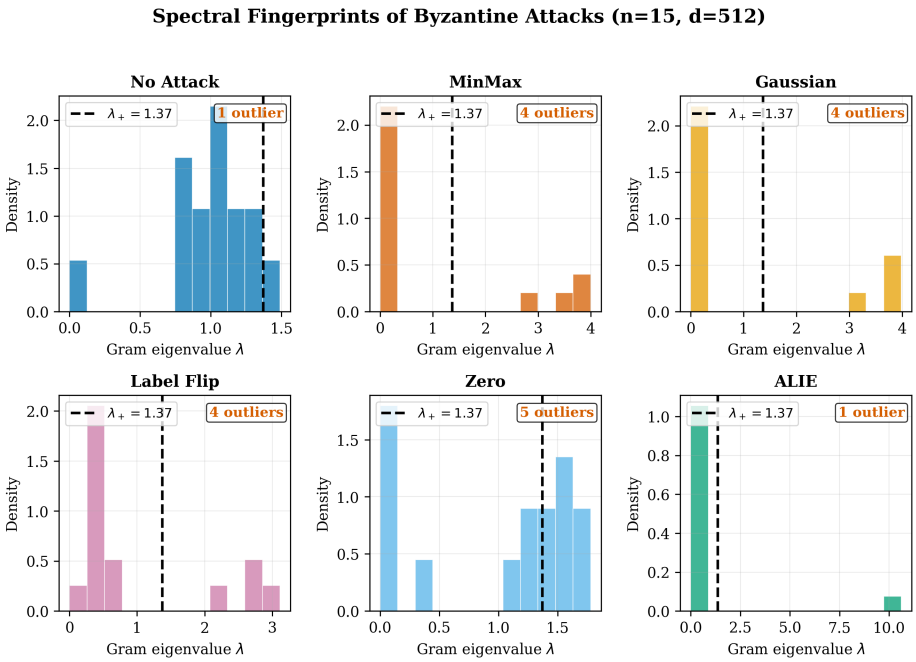

Fig. 6. Spectral fingerprints of six Byzantine attack types ($n = 15$, $d = 512$). The dashed line marks λ_+; eigenvalues to the right are Byzantine outliers. No-Attack shows no outliers (MP law holds); MinMax and Gaussian attacks create 4 outliers each; Zero-gradient (free-riding) creates 2 subtle outliers, yet all are detected by the KS test.

D.4 Memory Scalability

Full-covariance computation ($O(d^2)$ memory) becomes impractical above $d = 10^6$ parameters ($> 4\,\text{GB}$ at float32), while the sketched variant with $k = 256$ requires only $256^2 \cdot 4 \approx 262\,\text{MB}$—well within GPU memory limits up to $d = 10^9$. At the 345M-parameter scale (GPT-2-Medium), the $31\times$ memory reduction ($28\,\text{GB} \to 890\,\text{MB}$) enables deployment on a single A100 GPU.